SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	Manish Kumar L	**Reg. No.:**	1924260185
Date:	16/10/2026	**Duration:**	60 minutes

Code of Conduct

*I **MANISH KUMAR /1924260185** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: manish kumar

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Intelligent Digital Twin Monitoring Platform for Industrial Assets
Industry Context: Industrial organizations need continuous visibility into complex assets such as turbines, pumps, compressors, and production systems.
Problem Statement: Design and develop a digital-twin platform that maintains virtual representations of physical assets and synchronizes operational, maintenance, sensor, and performance information.

Objectives

- Create digital asset models.
- Synchronize sensor data.

- Monitor equipment status.
- Maintain maintenance history.
- Visualize asset performance.
- Identify abnormal conditions.
-

Expected Outcomes

- Improved asset visibility.
- Better maintenance planning.
- Faster fault identification.
- Reduced downtime.

Improved operational decision-making.

Architecture Design Analysis and Answer

1. System Requirements Analysis

The proposed platform creates a digital representation (digital twin) of each industrial asset and keeps it synchronized with information from physical equipment. It brings together live sensor readings, operating status, performance measures, and maintenance history so engineers and managers can monitor assets and make informed decisions.

System objectives:

- Create and maintain a digital model for each turbine, pump, compressor, or production system.
- Collect and synchronize sensor and operational data with the corresponding digital asset.
- Show current equipment status and historical performance.
- Store maintenance activities, inspections, and repair history.
- Detect abnormal readings or performance patterns and notify relevant users.
- Support maintenance planning and operational decisions through clear visualizations.

Functional requirements:

- Asset registration and digital model management, including asset identity, type, location, and key operating parameters.
- Sensor-data ingestion through device gateways or industrial protocols, with timestamps and asset identifiers.
- Data validation, normalization, storage, and association with the correct digital twin.
- Dashboards for equipment status, sensor trends, performance indicators, and asset comparisons.
- Maintenance-history management, including service dates, faults, actions taken, and upcoming maintenance.
- Abnormal-condition detection using thresholds or configurable rules, with alerts and event records.
- Role-based access for operators, maintenance engineers, and administrators.

Non-functional requirements:

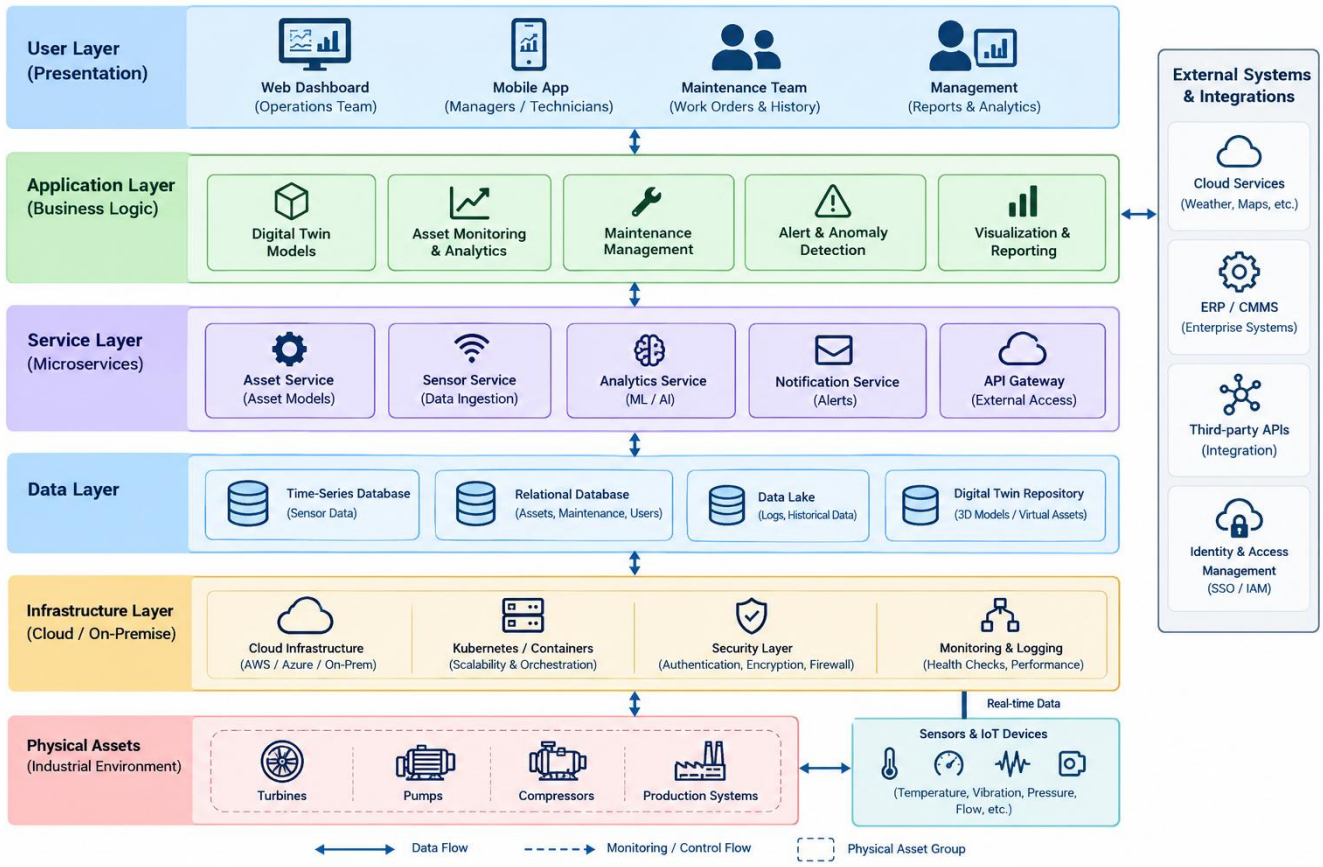
- Scalability: support growing numbers of assets, sensors, and users.
- Performance: process incoming readings promptly and keep dashboards responsive.
- Reliability and availability: tolerate component failures and avoid losing important readings or maintenance records.
- Security: authenticate users and devices, authorize access by role, encrypt data in transit, and audit important actions.
- Maintainability: keep components modular so individual services can be updated or replaced.
- Data integrity: validate readings, preserve timestamps and units, and track missing or invalid data.

2. Selection of Suitable Software Architecture

A hybrid architecture combining microservices, event-driven data processing, and a layered client-server design is recommended. Microservices separate major responsibilities (asset management, ingestion, analytics, maintenance, alerts, and visualization). An event-driven mechanism allows sensor readings to be processed asynchronously without forcing every component to communicate directly with every other component. The client-server interface provides dashboards through a web application and APIs.

This architecture fits the system because sensor data can arrive continuously and at high volume, while maintenance records and user requests follow different processing patterns. Independent services can scale according to workload, and APIs make it easier to integrate industrial gateways, enterprise maintenance systems, or future analytics tools.

3. Software Architecture Diagram



Cross-cutting services: identity and access management, encryption, audit logging, monitoring, backups, and configuration management apply across the architecture.

4. Components and Their Interactions

- Physical assets and sensors: Generate measurements such as temperature, vibration, pressure, speed, power, and operating state.
- Edge gateway and protocol adapters: Connect to industrial equipment, translate supported protocols into a common format, timestamp readings where needed, and buffer data during temporary network outages.
- Ingestion API: Validates incoming messages, checks asset identifiers and data format, and forwards accepted readings to the event stream.
- Message broker / event stream: Decouples data producers from consumers and distributes sensor events to the services that need them.

- Asset and digital-twin service: Maintains asset identity, configuration, model attributes, and the latest known state of each digital twin.
- Analytics and rule engine: Calculates performance indicators, evaluates thresholds or rules, identifies abnormal readings, and creates events for review.
- Maintenance service: Stores maintenance plans, inspection notes, repairs, service dates, and links these records to the relevant asset.
- Alert and event service: Creates alerts from abnormal-condition events and routes them to the appropriate user interface or notification channel.
- Data layer: Uses a time-series database for sensor history, a relational database for asset and maintenance records, and suitable storage for event and audit logs.
- Dashboard, APIs, and reporting: Present live status, trends, maintenance history, alerts, and reports; APIs enable controlled integration with other systems.

Typical workflow: a sensor sends a reading to the edge gateway; the gateway forwards it to the ingestion API; validated data is published to the event stream and stored. The digital-twin service updates the asset's current state, while analytics evaluates the reading. If a rule detects an abnormal condition, an event is created and the alert service makes it visible to the relevant users. Engineers can open the asset dashboard, review trends and maintenance history, and record follow-up work.

5. Quality Attributes

- Scalability: Deploy services independently and add ingestion or analytics instances as the number of assets and event volume increase. Partitioning the event stream and scaling storage helps handle growth.
- Security: Use authenticated device connections, role-based user access, encrypted communications, secrets management, input validation, and audit logs. Restrict external APIs and segment industrial networks.
- Performance: Use asynchronous event processing, time-series storage, efficient queries, caching of frequently viewed asset summaries, and suitable dashboard refresh intervals.
- Reliability: Buffer readings at the edge, use durable message queues, retry transient failures, monitor service health, and back up databases. Track gaps and duplicates in sensor data.
- Maintainability: Keep services focused on clear responsibilities, document APIs, automate tests and deployments, and use centralized logs and metrics to troubleshoot problems.
- Availability: Run critical services with redundancy, health checks, automated restart, database replication where appropriate, and recovery procedures. Define backup and disaster-recovery objectives.

6. Architectural Justification and Trade-offs

The hybrid approach is selected because it balances continuous sensor-data processing with asset management, maintenance workflows, and interactive dashboards. Event-driven communication prevents a slow analytics task from blocking sensor ingestion. Microservices let high-load functions such as ingestion or analytics scale separately, while the layered application and API design keeps user interfaces separate from business logic and data storage.

Trade-offs: microservices and event streaming introduce deployment, monitoring, security, and operational complexity compared with a single monolithic application. Asynchronous processing can also mean that dashboards show data with a small delay, and message retries may create duplicate events unless consumers are designed to handle them safely. For a

small initial deployment, a modular monolith with a message broker may be simpler; the platform can split services as scale and team needs grow.

Future enhancements can include predictive-maintenance models, remaining-useful-life estimates, 3D asset visualization, mobile alerts, integration with enterprise asset-management (EAM) or computerized maintenance-management systems (CMMS), and simulation-based what-if analysis. These additions can be introduced through documented APIs and new event consumers without redesigning the entire platform.

Conclusion: The proposed architecture provides a structured and extensible platform for synchronizing industrial assets with their digital twins. It supports asset visibility, maintenance planning, faster fault identification, reduced downtime, and better operational decision-making.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope;	Justification is weak or absent; report lacks organization and technical clarity.

	presents a well-organized, professional report.		report needs improvement.	

Criteria	Mark
System Requirement Analysis**(30%)**	/5
Selection of Software Architecture**(25%)**	/5
Architecture Diagram and Component Design**(20%)**	/5
Component Interaction and Quality Attribute Analysis**(15%)**	/5
Architectural Justification and Technical Presentation**(10%)**	/5
TOTAL	/25